V:N:M Sparsity for NEAT-Evolved Convolutional Networks

Athul Kesav (CIFAR-100, ImageNet-100) Anirudh (CIFAR-10)

Code: https://github.com/Athul-Kesav/codeepneat-vnm

1 Introduction

Neural architecture search by neuro-evolution finds working CNNs without hand design. CoDeepNEAT [1], for example, evolves graphs of modules and scores each candidate after a few epochs of training. The networks it finds are irregular: odd channel counts, large kernels, multi-way merges and, in its default search space, a large flatten+linear classifier. To deploy them, the weights must shrink and inference should get faster.

NVIDIA Sparse Tensor Cores accelerate only the 2:4 pattern, i.e. 50% sparsity [5]. VENOM [3] adds a vector level on top of 2:4. Its V:N:M format reaches 2:8, 2:16 and 2:32 (75–94% sparsity) and still feeds 2:4 tiles to the hardware through its Spatha kernel. However, VENOM has only been shown on transformers, and Spatha only multiplies matrices; it has no convolution.

Problem. Does V:N:M sparsity work on CNNs found by neuro-evolution, measured in accuracy, stored size and real inference time on Sparse Tensor Cores? And what properties of evolved networks help or hurt it?

Contributions.

(1) A path that applies V:N:M to every convolution and runs it on Spatha, checked against cuDNN on all 312 timed configurations.
(2) An evaluation on CIFAR-10, CIFAR-100 and ImageNet-100 over 10 evolved models, the full $V \times M$ grid and three pruning criteria, with ResNets as a brief reference.
(3) A CoDeepNEAT search-space change (residual sum + global average pooling) that makes evolved networks compressible: whole-model compression goes from 1.3× to 24×, and dense accuracy rises by 4 pt.
(4) Findings specific to evolved networks:
   - irregular widths clash with large V;
   - gradual second-order pruning decides high sparsity;
   - the recovery recipe decides what looks feasible;
   - evolved wide convolutions suit Spatha better than thin ResNets;

   as well as two silent Spatha failure modes and an accuracy bug in CoDeepNEAT's evaluator.

2 Prior work

- **NEAT and CoDeepNEAT.** CoDeepNEAT [1] co-evolves two populations: *modules*, which are small graphs of conv, BN, pooling and dropout layers, and *blueprints*, which are graphs whose nodes are filled with a module from a module species. NEAT mutations add nodes and connections, and speciation protects new structure. We build on the PyTorch implementation by `sash-a` [2]. There, branches merge by channel concatenation and the classifier is flatten→linear.
- **N:M sparsity.** N:M sparsity keeps N of every M consecutive weights. Hardware supports only 2:4 [5]. It has been trained into CNNs at 2:4-like densities [6].
- **VENOM and Spatha** [3, 4].
  - Format: a weight matrix is tiled into blocks of V rows × M columns. Each block keeps the same 4 columns for all V rows (vector level), and each row keeps 2 of those 4 (2:4 level), so density is $2/M$.
  - Kernel: Spatha gathers the kept columns of the dense operand and issues 2:4 `mma.sp` instructions.
  - Pruning: VENOM reached high sparsity on BERT and GPT models with gradual second-order pruning.
- **Second-order pruning.**
  - Optimal Brain Surgeon (OBS) [7] removes weight w_c at cost $w_c^2/[H^{-1}]_{cc}$ and corrects the remaining weights by $\delta w = -\frac{w_c}{[H^{-1}]_{cc}} H^{-1}_{:,c}$.
  - SparseGPT [8] applies this one-shot and layer-wise, with $H = \frac{2}{N}\sum xx^\top$.
  - oBERT [9] applies it gradually, with fine-tuning between steps.
- **Starting point for CIFAR-10.** The CIFAR-10 part of this project started from a teammate's study [12]. It provided the CoDeepNEAT searches, a Python V:N:M prune/pack path, and an 8-epoch recovery fine-tune, but no Sparse Tensor Core execution.

3 Idea and implementation

Pipeline.

(i) Evolve with CoDeepNEAT.
(ii) Retrain the top 3 architectures dense, with a modern recipe.
(iii) Prune every convolution to V:2:M, with $M \in \{4, 8, 16, 32\}$ and $V \in \{8, 16, 32, 64, 128\}$.

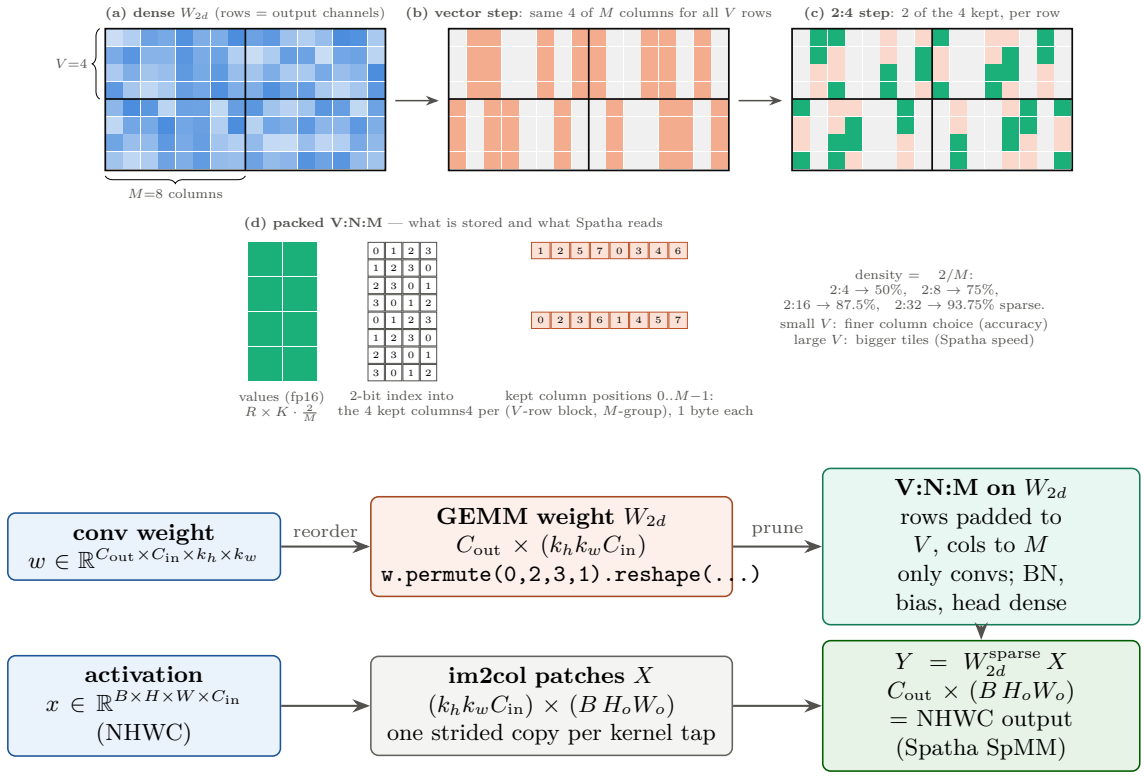


Figure 1: Top: V:N:M on one $V \times M$ block. The vector step keeps 4 columns for all V rows, and the 2:4 step keeps 2 per row. Bottom: a convolution is reordered to a $C_{\text{out}} \times k_h k_w C_{\text{in}}$ matrix, pruned, and multiplied with im2col patches by Spatha.

(iv) Fine-tune with the mask fixed.
(v) Store the pruned weights in VENOM's packed format and run inference on Spatha.

BN, biases and the classifier stay dense in fp32.

3.1 Convolutions as V:N:M matrices

- **Layout.** We prune $W_{2d} = w.\text{permute}(0,2,3,1).\text{reshape}(C_{\text{out}}, k_h k_w C_{\text{in}})$. Input channels are innermost, so a column group is M input channels at one kernel tap, and the vector step drops (tap, channel) inputs for a group of V output channels (Fig. 1).
- **Storage.** fp16 values, a 2-bit index per value, and one byte per kept column. Rows are padded to a multiple of V and K to a multiple of M, and the padding is counted in every size we report.
- **Inference.** `SpathaConv2d` builds the im2col matrix with one strided copy per kernel tap (channels-last), calls `spatha.spmm`, and uses the output directly as NHWC.
- **Spatha builds.** We built Spatha for $V = 32, 64, 128$ and 256; $V = 32$ and 256 need a one-hunk tile patch. We found two undocumented failure modes, both of which return wrong results without raising an error:
  – at M=32, K must be padded to a multiple of 256, otherwise there is an out-of-bounds write;
  – one call handles at most 65,535 column tiles, so large inputs are chunked.

3.2 A V:N:M-friendly search space

In CoDeepNEAT's default space, the flatten+linear head held 34–77% of all bytes, and V:N:M cannot prune it. We added two options to CoDeepNEAT and searched CIFAR-100 again with 10-epoch evaluations:

- **residual-sum aggregation:** inputs at the widest channel count pass through, and narrower ones get a 1×1 projection;
- **a global-average-pool head.**

Other changes to CoDeepNEAT:

- **Evaluator fix.** `test_nn` divided the summed batch accuracies by $n-1$ batches, which overstated every reported accuracy by 1.3–2.6%. We fixed it and corrected the affected numbers.
- **Robust search.** A blueprint that runs out of GPU memory now scores 0 instead of crashing the generation.
- **New loaders:** CIFAR-100 and ImageNet-100 (the CMC 100-class split [10], 128^2 crops).

	Colab pilot	CIFAR-10 small	CIFAR-10 strong	CIFAR-100 default	CIFAR-100 res+GAP	ImageNet-100 default
generations	12	12	**80**	12	12	10
evaluations × epochs per blueprint	1 × 2	2 × 3	4 × 12	2 × 3	2 × 10	2 × 3
aggregation / head	concat / flatten+linear		(res+GAP: residual sum / GAP+linear)			concat / flatten
hardware, wall-clock	1 T4	3 Ada, 1.7 h	2 Ada, ≈2 days	3 Ada, 1.7 h	3 Ada, 12.8 h	3 Ada, 2.4 h

Table 1: The six CoDeepNEAT searches. The Colab pilot and the strong CIFAR-10 search are the teammate's.

dataset, best evolved model		dense	2:4	2:8	2:16	2:32
CIFAR-10, bp1485 (11.1M params, strong search)	acc. %	86.45†	86.35	**86.52**	85.48	81.55*
	size (MB)	44.4	14.0	7.0	3.5	1.8
CIFAR-100, bp493 (8.4M, residual+GAP search)	acc. %	76.50	76.26	74.40*	70.50*	63.72*
	size (MB)	33.7	10.8	5.4	2.8	1.4
ImageNet-100, bp207 (2.8M, default space)	acc. %	53.68	**54.30**	53.28	48.00	38.70
	size (MB)	11.1	8.7	8.2	7.9	7.8

Table 2: Headline results: test accuracy and whole-model stored size (at V=8) after V:N:M pruning and the 30-epoch fine-tune. Accuracy is best over V (usually 8); magnitude pruning unless marked * (gradual OBS). † Dense control; the dense checkpoint scores 85.61%. Conv compression is about 3.1/6.3/12.6/25× at 2:4/2:8/2:16/2:32. The ImageNet-100 model barely shrinks because its dense flatten head is not pruned.

3.3 Training, recovery and pruning criteria

- **Dense retraining.** The evolved architectures are kept and retrained from scratch.
  - Recipe v1: AdamW, cosine schedule, label smoothing, crop/flip, bf16; 200 epochs on CIFAR-100 and 120 on ImageNet-100.
  - Recipe v2 (CIFAR-100): v1 for 300 epochs, plus CutMix and EMA.

  CIFAR-10 uses CoDeepNEAT's own 150-epoch full train.
- **Recovery after pruning.** 30 epochs of AdamW with distillation from the dense model (α=0.5, T=4) and crop/flip. The mask is re-applied after every optimizer step.
- **Criteria.**
  - *Magnitude.*
  - *One-shot OBS*: SparseGPT-style, with masks that respect the V:N:M block structure, plus the OBS weight update.
  - *Gradual OBS*, as in VENOM/oBERT: the target M is reached through intermediate column steps, each with a fresh Hessian and a short fine-tune, followed by the full 30-epoch fine-tune.
- **Timing.** Spatha against *dense-best*: per layer, the faster of cuDNN and the same im2col path with a dense fp16 GEMM. Medians of 5 interleaved rounds on RTX 6000 Ada GPUs (sm_89).

4 Evaluation against the baselines

Baselines.

- *Dense evolved network:* the accuracy reference. On CIFAR-10 we also use a dense *control*, given the identical 30-epoch fine-tune, because that fine-tune alone lifts accuracy.
- *Magnitude pruning and CoDeepNEAT's default search space:* the pruning and architecture baselines.
- *Dense-best inference:* the speed baseline.
- *Size-matched ResNets* [11] (ResNet-18, ResNet-20 width 58, ResNet-56 width 32), trained with the same recipe: a brief check.

4.1 CIFAR-10: the recovery recipe decides what looks feasible

The strong search's winner bp1485 has six 3×3 convs (329/624 channels). Its max-pools leave a 4×4 map, so 99.9% of its bytes are convolutions.

- **Against the dense control,** 2:8 loses nothing and 2:16 loses 1 pt, at 12.6× conv compression (44.4 → 3.5 MB).

recovery after pruning	V	2:4	2:8	2:16	2:32
none (zero-shot)	8	41.11	16.53	10.97	10.00
8-ep Adam, no augmentation, no KD (original study)	8	86.01	79.78	26.36	–
30-ep KD fine-tune, magnitude	8	86.30	**86.52**	**85.48**	79.11
30-ep KD fine-tune, magnitude	32	**86.35**	86.36	85.02	80.52
gradual OBS	8	–	–	84.42	**81.55**
dense checkpoint 85.61%; dense control (same 30-epoch fine-tune, no pruning) **86.45%**					

Table 3: CIFAR-10, bp1485: test accuracy (%) under the original 8-epoch recipe and ours.

family	criterion	model	params	dense	2:4	2:8	2:16	2:32
evolved res+GAP	magnitude	bp493	8.42M	76.50	76.26	74.01	70.06	57.23
	one-shot OBS				–	73.92	69.23	57.98
	gradual OBS				–	**74.40**	**70.50**	**63.72**
evolved res+GAP	magnitude / gradual	bp741	4.33M	72.47	72.20	70.15 / **70.92**	66.60 / **68.09**	57.05 / **61.78**
evolved res+GAP	magnitude / gradual	bp482	2.51M	72.69	72.44	69.60 / **70.24**	62.14 / **64.33**	42.37 / **55.37**
evolved, default space	magnitude	bp430	3.62M	72.39	71.34	68.40	62.40	49.79

Table 4: CIFAR-100 test accuracy (%), best over V, after the 30-epoch fine-tune (35–45 epochs for gradual OBS). Shaded: best evolved result per column.

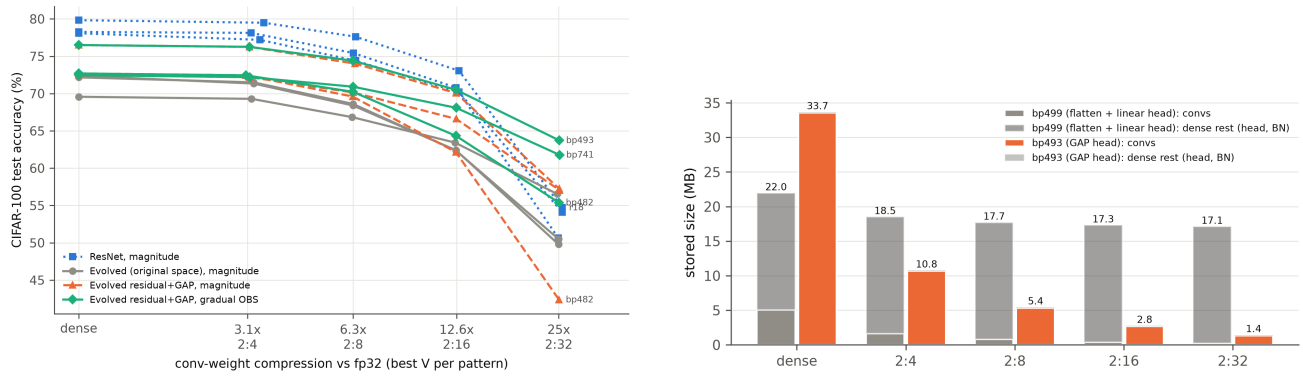


Figure 2: Left: CIFAR-100 accuracy vs. conv compression. Gradual OBS (aqua) lifts the evolved residual models above every magnitude-pruned curve at 25×, including the ResNets. Right: stored size at V=8. bp499 (default space) keeps a 17 MB dense head, while bp493 (GAP head) shrinks with the conv ratio.

The original 8-epoch recovery made 2:16 look unusable (26%), so the recipe, not V:N:M, was the limit.

- V **hardly matters** on these wide layers (within 0.5 pt up to 2:16).
- **Gradual OBS helps only at 2:32** (+2.4 pt).
- **The Colab pilot** (bp608, 81.98%) shows the head problem in its extreme form: its convolutions hold only 4.8% of its 47 MB, so V:N:M shrinks it by only 1.05×.

4.2 CIFAR-100: accuracy, size and the pruning criterion

- **Training before search.** Retraining the same evolved architectures lifted them from at most 50% (CoDeepNEAT's own training) to 72.4%. The residual+GAP search added 4 pt more (76.5%).
- **Accuracy against dense.** 2:4 costs 0.24–1.05 pt on every evolved model. 2:8 costs 1.6–2.5 pt with gradual OBS, and 2:16 costs 4.4–8.4 pt. Degradation is smooth.
- **Size against the default search space.** In the default space, conv compression barely becomes model compression: about 1.3× even at 2:32. With residual+GAP, over 99% of the bytes are prunable, and 33.7 MB becomes 5.4 MB at 2:8 and 1.4 MB at 2:32 (Fig. 2, right). Below about 1 MB, gradual-OBS evolved models form the size–accuracy Pareto front.
- **Gradual OBS against magnitude.** Averaged over 3 models × $V \in \{8, 32, 128\}$, it gains +0.62 / +2.48 / **+9.18** pt at 2:8 / 2:16 / 2:32 and wins 26 of 27 comparisons. One-shot OBS gives a far better zero-shot start (22–49% instead of about 1% at 2:8), but fine-tuning erases that lead. Entering the sparse subspace step by step is what helps.
- **Width and V.** Smaller V is more accurate: V=128 costs 1–3 pt at 2:8 and 3–5 pt at 2:16, compared with V=8. Irregular evolved widths (142, 171, 211, …) are also padded up to V rows, which cuts 2:8 conv compression from 6.3× to 4.1–5.7× at V=128. Power-of-two ResNet widths instead gain compression at large V.
- **ResNets.** Size-matched ResNets are 1.6–1.8 pt more accurate when dense and lose about the same under V:N:M. At 2:32, gradual OBS on the evolved models (63.7%) beats every magnitude-pruned ResNet (54.7%).

4.3 ImageNet-100

bp207 / bp182 / bp230 (2.77M / 3.05M / 0.47M parameters) were retrained for 120 epochs, reaching 53.68 / 52.36 / 52.28%. CoDeepNEAT's own training had reached only about 29%.

- 2:4 is near-lossless: at most 0.5 pt, and bp207 even gains 0.6 pt because the distilled fine-tune acts as extra training.
- 2:8 costs 0.4–3 pt, and accuracy degrades gracefully to 38.7–41.6% at 2:32.
- These models come from the default search space, so the whole model shrinks only 1.2–1.4× (2.0–3.4× for bp230), even though the convolutions shrink 3–25×.

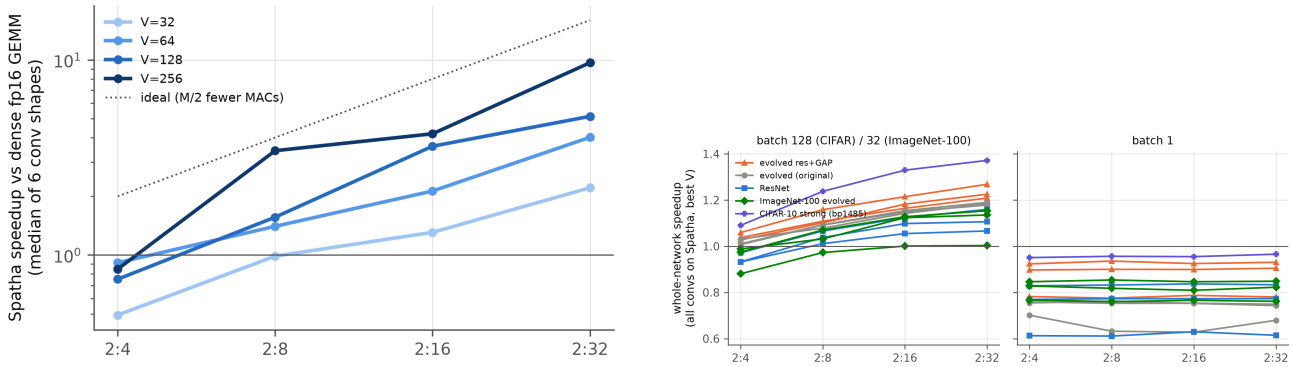


Figure 3: Left: Spatha conv-GEMM speedup over dense fp16 GEMM by pattern and V. Right: whole-network speedup with every conv on Spatha, against dense-best, at large batch and at batch 1.

dataset	model	dense-best (ms)	2:4	2:8	2:16	2:32
CIFAR-10	bp1485	22.6	1.09	1.24	1.33	**1.37**
CIFAR-100	bp493 / bp741	24.3 / 16.4	1.03 / 1.06	1.10 / 1.16	1.18 / 1.21	1.23 / 1.27
CIFAR-100	ResNet-18 / ResNet-56 (w32)	17.7 / 29.6	0.97 / 0.93	1.07 / 1.01	1.12 / 1.05	1.16 / 1.07
ImageNet-100	bp207 / bp182	14.4 / 12.5	0.97 / 0.99	1.07 / 1.03	1.13 / 1.12	1.15 / 1.14
all models, batch 1			0.50–0.94 (slower)			

Table 5: End-to-end speedup of a full forward pass, best over $V \in \{32, 64, 128\}$, at batch 128 (CIFAR) or 32 (ImageNet-100). Every one of the 312 runs agrees with cuDNN (top-1 agreement $\geq$99.2%).

4.4 Speed on Sparse Tensor Cores

- **GEMM level.** Spatha is 1.4–2.6× faster than dense at 2:8 and up to about 8× at 2:32, and the gain grows with V. At 2:4 it is slower than dense for most V.
- **Whole network.** Only 1.0–1.37× faster at large batch, and slower at batch 1, where it is overhead-bound. This is Amdahl's law applied twice:
  - convolutions are only 34–60% of the forward time;
  - within each convolution, the im2col copy costs as much as the sparse GEMM.

  For bp493 at 2:32, for example, the GEMM is about 8× faster, but im2col + Spatha is only 1.6× faster than im2col + dense GEMM.
- **Evolved vs. ResNets.** Evolved networks gain more than ResNets (1.27–1.37× vs. 1.16×): short-budget evolution produces shallow, wide, large-kernel convs, whose GEMMs are large. The thin ResNet-56 barely speeds up.
- **Accuracy against speed.** The two favour opposite V: accuracy is best at V=8, but Spatha exists only for $V \geq 32$ and is fastest at V=128.

4.5 Discussion, limitations and conclusion

V:N:M works on NEAT-evolved CNNs as compression. 2:4 is near-lossless on all 15 well-trained CNNs (median drop 0.3 pt, at most 1.05 pt), and 2:8 costs 0–2.1 pt on the best models. Three conditions made it work:

- a search space that puts the parameters into convolutions;
- a long, distilled fine-tune;
- at high sparsity, gradual second-order pruning.

Speed is the weak part. Turning 4–8× GEMM speedups into network speedups needs an implicit-GEMM sparse convolution and fused epilogues. A V:N:M-aware search should score candidates after pruning and constrain widths to multiples of V.

Limitations.

- Each result is a single seed, and models and V were selected on the test set.
- Gradual OBS had 35–45 fine-tune epochs against 30 for the other criteria, and there is no gradual-magnitude control.
- OBS was not run on the ResNets.
- Timing used shared GPUs and eager PyTorch 1.13, and V=8/16 cannot be timed.

References

[1] R. Miikkulainen et al., "Evolving deep neural networks," 2019. https://arxiv.org/abs/1703.00548

[2] sash-a, CoDeepNEAT implementation in PyTorch. https://github.com/sash-a/CoDeepNEAT

[3] R. L. Castro, A. Ivanov, D. Andrade, T. Ben-Nun, B. B. Fraguela, T. Hoefler, "VENOM: A vectorized N:M format for unleashing the power of sparse tensor cores," SC'23. https://arxiv.org/abs/2310.02065

[4] VENOM / Spatha code. https://github.com/UDC-GAC/venom

[5] A. Mishra et al., "Accelerating sparse deep neural networks," 2021. https://arxiv.org/abs/2104.08378

[6] A. Zhou et al., "Learning N:M fine-grained structured sparse neural networks from scratch," ICLR 2021. https://arxiv.org/abs/2102.04010

[7] B. Hassibi, D. G. Stork, "Second order derivatives for network pruning: Optimal Brain Surgeon," NeurIPS 1992. https://proceedings.neurips.cc/paper/1992/hash/303ed4c69846ab36c2904d3ba8573050-Abstract.html

[8] E. Frantar, D. Alistarh, "SparseGPT: Massive language models can be accurately pruned in one-shot," ICML 2023. https://arxiv.org/abs/2301.00774

[9] E. Kurtic et al., "The Optimal BERT Surgeon," EMNLP 2022. https://arxiv.org/abs/2203.07259

[10] Y. Tian, D. Krishnan, P. Isola, "Contrastive multiview coding" (ImageNet-100 split), ECCV 2020. https://arxiv.org/abs/1906.05849

[11] K. He et al., "Deep residual learning for image recognition," CVPR 2016. https://arxiv.org/abs/1512.03385

[12] Anirudh, CIFAR-10 CoDeepNEAT + V:N:M study. https://github.com/Anirudhmoosad/codeepneat-venom

[13] This project's code. https://github.com/Athul-Kesav/codeepneat-vnm